

The Role of Permutation Invariance in Linear Mode Connectivity of Neural Networks

Rahim Entezari¹, Hanie Sedghi², Olga Saukh¹, and Mehran Neyshabur³

¹TU Graz / CSH Vienna

²Google Research, Brain Team

³Google Research, Shift Team

Abstract

In this paper, we conjecture that if the permutation invariance of neural networks is taken into account, SGD solutions will likely have no barrier in the linear interpolation between them. Although it is a bold conjecture, we show how extensive empirical attempts fall short of refuting it. We further provide a preliminary theoretical result to support our conjecture. Our conjecture has implications for lottery ticket hypothesis, distributed training and ensemble methods.

1 Introduction

Understanding the loss landscape of deep neural networks has been the subject of many studies due to its close connections to optimization and generalization (Li et al., 2017; Mei et al., 2018; Geiger et al., 2019; Nguyen et al., 2018; Fort et al., 2019; Baldassi et al., 2020). Empirical observations suggest that loss landscape of deep networks has many minima (Keskar et al., 2017; Draxler et al., 2018; Zhang et al., 2017). One reason behind the abundance of minima is over-parametrization. Over-parametrized networks have enough capacity to present different functions that behave similarly on the training data but vastly different on other inputs (Neyshabur et al., 2017; Nguyen et al., 2018; Li et al., 2018; Liu et al., 2020). Another contributing factor is the existence of scale and permutation invariances which allows the same function to be represented with many different parameter values of the same network and imposes a counter-intuitive geometry on the loss landscape (Neyshabur et al., 2015; Brea et al., 2019a).

Previous work study the relationship between different minima found by SGD and establish that they are connected by a path of non-increasing loss; however, they are *not* connected by a *linear* path (Freeman and Bruna, 2016; Draxler et al., 2018; Garipov et al., 2018). This phenomenon is often referred to as mode connectivity (Garipov et al., 2018) and the loss increase on the path between two solutions is often referred to as (energy) barrier (Draxler et al., 2018). Nagarajan and Kolter (2019) is probably the first example who considered linear mode connectivity of trained MLPs from the same initialization. They note that the resulting networks are connected by linear paths of constant test error. Understanding *linear mode connectivity* (LMC) is highly motivated by several direct conceptual and practical implications from pruning and sparse training to distributed optimization and ensemble methods.

The relationship between LMC and pruning was established by Frankle et al. (2020) where they showed the correspondence between LMC and the well-known lottery ticket hypothesis (LTH) (Frankle and Carbin, 2019). In short, LTH conjectures that neural networks contain sparse subnetworks that can be trained in isolation, from initialization, or early in training to achieve comparable test accuracy. Frankle et al. (2020)

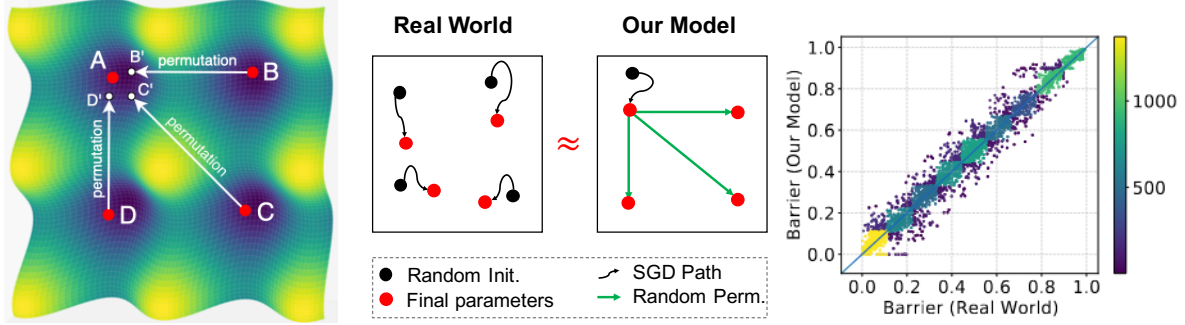


Figure 1: **Linear mode connectivity when using permutation invariance.** **Left:** Schematic picture of four minima A, B, C, D in different basins with an energy barrier between each pair. However, our conjecture suggests that permuting hidden units of B, C and D would result in B', C' and D' which present the exact same function as before permutation while having no barrier on their linear interpolation with A . **Middle:** Our model for barriers in real world SGD solutions. In real world we train networks by running SGD with different random seeds starting from different initializations. In our model, different final networks are achieved by applying random permutations to the same SGD solution (or equivalently, applying random permutations to the same initialization and then running SGD with the same seed on them). **Right:** aggregation of our extensive empirical evidence (more than 3000 trained networks) in one density plot comparing barriers in real world against our model across different choices of architecture family, dataset, width, depth, and random seed. Points in the lower left mostly correspond to lowest barrier found after searching in the space of valid permutations using a Simulated annealing (S). For a detailed view on architectures and datasets see Figure 10 in appendix 3.

showed that solutions that are linearly connected with no barrier have the same lottery ticket. They further discuss how linear-connectivity is associated with stability of SGD. This view suggests that SGD solutions that are linearly connected with no barrier can be thought of as being in the same basin of the loss landscape and once SGD converges to a basin, it shows a stable behavior inside the basin¹. Because of the direct correspondence between LMC and LTH, any understanding of LMC, has implications for LTH, stability of SGD and pruning techniques.

Linear mode connectivity has also direct implications for ensemble methods and distributed training. Ensemble methods highly depend on an understanding of the loss landscape and being able to sample from solutions. Better understanding of mode connectivity has been shown to be essential in devising better ensemble methods (Garipov et al., 2018). Linear mode connectivity between solutions or checkpoints also allows for weight averaging techniques for distributed optimization to be used as effectively in deep learning as convex optimization (Scaman et al., 2019).

In this paper, we conjecture that by taking permutation invariance into account, the loss landscape can be simplified significantly resulting in linear mode connectivity between SGD solutions. We investigate this conjecture both theoretically and empirically through extensive experiments. We show how our attempts fall short of refuting this hypothesis and end up as supporting evidence for it (see Figure 1). We believe our conjecture sheds light into the structure of loss landscape and could lead to practical implications for the aforementioned areas².

Contributions. This paper makes the following contributions:

- We study linear mode connectivity (LMC) between solutions trained from different initializations and investigate how it is affected by choices such as width, depth and task difficulty for fully connected and convolutional networks (Section 2).
- We introduce our main conjecture in Section 3: If invariances are taken into account, there will likely be no barrier on the linear interpolation of SGD solutions (see the left panel of Figure 1).

¹This notion of basin is also consistent with the definition proposed by Neyshabur et al. (2020).

²The source code is available at <https://github.com/rahimentezari/PermutationInvariance>

- By investigating the conjecture theoretically, we prove that it holds for a wide enough fully-connected network with one hidden layer at random initialization (Section 3).
- In Section 4, we provide strong empirical evidence in support of our conjecture. To overcome the computational challenge of directly evaluating the hypothesis empirically, which requires searching in the space of all possible permutations, we propose an alternative approach. We consider a set of solutions corresponding to random permutations of a single fixed SGD solution (our model) and show several empirical evidences suggesting our model is a good approximation for all SGD solutions (real world) with different random seeds (see the middle and right panel of Figure 1).

Further related work. Permutation symmetry of neurons in every layer results in multiple equivalent minima connected via saddle points. Few studies investigate the role of these symmetries in the context of connectivity of different basins.

Given a network with L layers of minimal widths $r_1^*, \dots, r_{L-1}^*$ that reaches zero-loss minima at $r_1!, \dots, r_{L-1}!$ isolated points (permutations of one another), Şimşek et al. (2021) showed that adding one extra neuron to each layer is sufficient to connect all these previously discrete minima into a single manifold. Fukumizu and mari (2000) prove that a point corresponding to the global minimum of a smaller model can be a local minimum or a saddle point of the larger model. Brea et al. (2019b) find smooth paths between equivalent global minima that lead through a permutation point, *i.e.*, where the input and output weight vectors of two neurons in the same hidden layer interchange. They describe a method to permute all neuron indices in the same layer at the same cost. Tatro et al. (2020) showed that aligning the neurons in two different neural networks makes it easier to find second order curves between them in the loss landscape where barriers are absent.

2 Loss Barriers

In this section, we first give a formal definition for linear mode connectivity and study how it is affected by different factors such as network width, depth, and task difficulty for a variety of architectures.

2.1 Definitions

Let $f_\theta(\cdot)$ be a function presented by a neural network with parameter vector θ that includes all parameters and $\mathcal{L}(\theta)$ be the any given loss (e.g., train or test error) of $f_\theta(\cdot)$. Let $\mathcal{E}(\theta_1, \theta_2) = \mathcal{L}(\theta_1 + (1 - \alpha)\theta_2)$, for $\alpha \in [0, 1]$ be the loss of the network created by linearly interpolating between parameters of two networks $f_{\theta_1}(\cdot)$ and $f_{\theta_2}(\cdot)$. The loss barrier $B(\theta_1, \theta_2)$ along the linear path between θ_1 and θ_2 is defined as the highest difference between the loss occurred when linearly connecting two points θ_1, θ_2 and linear interpolation of the loss values at each of them:

$$B(\theta_1, \theta_2) = \sup[\mathcal{L}(\theta_1 + (1 - \alpha)\theta_2)] - [\mathcal{L}(\theta_1) + (1 - \alpha)\mathcal{L}(\theta_2)]. \quad (1)$$

The above definition differs from what was proposed by Frankle et al. (2020) in that they used $0.5\mathcal{L}(\theta_1) + 0.5\mathcal{L}(\theta_2)$ instead of $\mathcal{L}(\theta_1) + (1 - \alpha)\mathcal{L}(\theta_2)$ in our definition. These definitions are the same if $\mathcal{L}(\theta_1) = \mathcal{L}(\theta_2)$. But if $\mathcal{L}(\theta_1), \mathcal{L}(\theta_2)$ are different, we find our definition to be more appropriate because it assigns no barrier value to a loss that is changing linearly between θ_1 and θ_2 .

We say that two networks θ_1 and θ_2 are linear mode connected if the barrier between them along a linear path is ≈ 0 (Frankle et al., 2020). It has been observed in the literature that any two minimizers of a deep network can be connected via a non-linear low-loss path (Garipov et al., 2018; Draxler et al., 2018; Fort and Jastrzebski, 2019). This work examines *linear* mode connectivity (LMC) between minima.

Next, we empirically investigate the effect of task difficulty and choices such as architecture family, width and depth on LMC of SGD solutions.

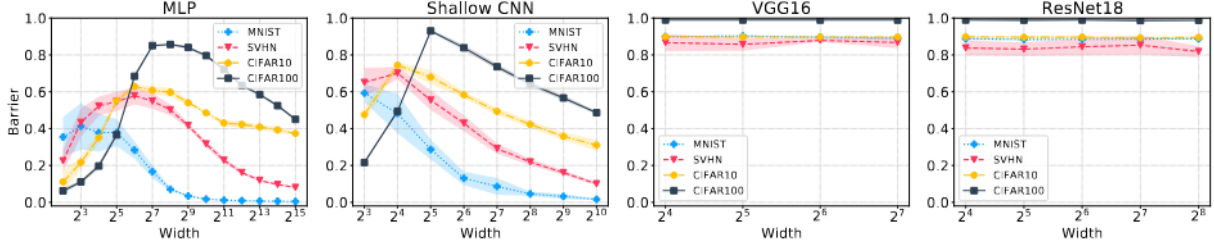


Figure 2: **Effect of width on barrier size.** From left to right: one-layer MLP, two-layer Shallow CNN, VGG-16 and ResNet-18 architectures on MNIST, CIF R-10, SVHN, CIF R-100 datasets. For large width sizes the barrier becomes small. This effect starts at lower width for simpler datasets such as MNIST and SVHN compared to CIF R datasets. closer look reveals a similar trend to that of double-descent phenomena. MLP architectures hit their peak at a lower width compared to CNNs and a decreasing trend starts earlier. For VGG and ResNet, the barrier size is saturated at a high value and does not change due to the effect of depth as discussed in Figure 3.

2.2 Empirical Investigation: Barriers

In this section, we look into barriers between different SGD solutions on all combinations of four architecture families (MLP (Rosenblatt, 1961), Shallow CNN (Neyshabur, 2020), ResNet (He et al., 2015) and VGG (Simonyan and Zisserman, 2015)) and four datasets (MNIST (LeCun and Cortes, 2010), SVHN (Netzer et al., 2011), CIF R-10 (Krizhevsky et al., 2009) and CIF R-100 (Krizhevsky et al., 2009)). The main motivation to use Shallow CNN is to move from fully connected layers (MLP) to convolutions. The main difference between Shallow CNN and VGG16 is depth and the main difference between ResNet18 and VGG16 is existence of residual connections.

We empirically investigate how different factors such as architecture family, width, depth and task difficulty impact the barrier size³. We refer to training loss barrier as barrier. For loss barriers on a test set see E.4. For train and test errors see .2 .

Width: We evaluate the impact of width on the barrier size in Figure 2. We note that for large values of width the barrier becomes small. This effect starts at lower width for simpler datasets such as MNIST and SVHN compared to CIF R datasets. closer look reveals that the barrier increases with width up to a point and beyond that increasing width leads to lower barrier size. This effect is reminiscent of the double descent phenomena (Belkin et al., 2019; Nakkiran et al., 2019). Checking the test error (Figure 8) indicates that in our experiments the barrier peak happens at the same size that needed to fit the training data. This phenomena is observed for both fully-connected and convolutional architectures. MLP architectures hit their peak at a lower width compared to CNNs and a decreasing trend starts earlier. For ResNets the barrier size is saturated at a high value and does not change. The barrier value for VGG architecture on different datasets is also saturated at a high value and does not change by increasing the width. Such similar behavior observed for both ResNets and VGG architectures is due to the effect of depth as discussed in the next paragraph.

Depth: We vary network depth in Figure 3 to evaluate its impact on the barrier between optimal solutions obtained from different initializations. For MLPs, we fix the layer width at 2^{10} while adding identical layers as shown along the x-axis. We observe a fast and significant barrier increase as more layers are added. For VGG architecture family we observe significant barriers. This might be due to the effect of convolution or depth. In order to shed light on this observation, we use Shallow CNN (Neyshabur, 2020) with only two convolutional layers. s can be seen in Figure 3 when Shallow CNN has two layers the barrier size is low, while keeping the layer width fixed at 2^{10} and adding more layers increases the barrier size. For residual

³In all plots the barrier is evaluated across 5 different random pairs (10 random SGD solutions). In our experiments, we observed that evaluating the barrier at $= \frac{1}{2}$ is a reasonable surrogate for taking the supremum over (the difference is less than $1 - 4$). Therefore, to save computation, we report the barrier value at $= \frac{1}{2}$.

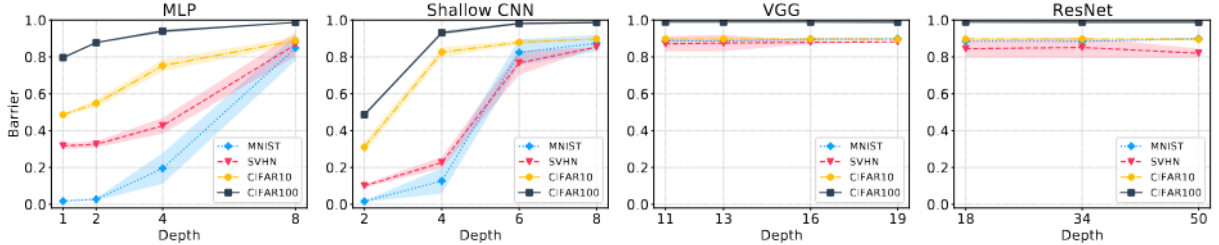


Figure 3: **Effect of depth on barrier size.** From left to right MLP, Shallow CNN, VGG(11,13,16,19), and ResNet(18,34,50) architectures on MNIST, CIF R-10, SVHN, CIF R-100 datasets. For MLP and Shallow CNN, we fix the layer width at 2^1 while adding identical layers as shown along the x-axis. Similar behavior is observed for fully-connected and CNN family, *i.e.*, low barrier when number of layers are low while we observe a fast and significant barrier increase as more layers are added. Increasing depth leads to higher barrier values until it saturates (as seen for VGG and ResNet).

networks we also consider three ResNet architectures with 18, 34 and 50 layers and observe the same barrier sizes as VGG for all these depth values. The main overall observation from depth experiments is that for both fully-connected and convolutional architectures, increasing depth increases the barrier size significantly so the effect of depth is not similar to width. This can also be attributed to the observations that deeper networks usually have a less smooth landscape (Li et al., 2017).

Task difficulty and architecture choice: In Figure 4 we look into the impact of the task difficulty provided by the dataset choice (MNIST, SVHN, CIF R-10, CIF R-100, and ImageNet (Deng et al., 2009)) and the architecture type (one-layer MLP with 2^{10} neurons, Shallow CNN with two convolutional layer and width of 2^{10} , VGG-16 with batch-normalization, ResNet18 and ResNet50). Each row in Figure 4a and Figure 4b shows the effect of task difficulty, *e.g.*, fixing the task to SVHN and moving from MLP to Shallow CNN gives lower test error hence lower barrier size. Each column also represents the effect of architecture on a specific dataset, *e.g.*, fixing the architecture to Shallow CNN and moving from CIF R10 to CIF R100 presents an increase in test error, hence increase in the barrier size. Although deep architectures like VGG16 and ResNet18 present low test error, the discussed effect of depth saturates their barrier at a high level. Figure 4c aggregates the correlation between test error and size of the barrier. For MLP and Shallow CNN we observe a high positive correlation between test error and barrier size across different datasets. Deeper networks (VGGs, ResNets) form a cluster in the top-left, with low test error and high barrier size.

3 Role of Invariance in Loss Barriers

Understanding the loss landscape of deep networks has proven to be very challenging. One of the main challenges in studying the loss landscape without taking the optimization algorithm into account is that there exist many minima with different generalization properties. Most of such minima are not reachable by SGD and we only know about their existence through artificially-made optimization algorithms and training regimes (Neyshabur et al., 2017). To circumvent this issue, we focus on parts of the landscape that are reachable by SGD. Given a dataset and an architecture, one could define a probability distribution over all solutions reachable by SGD and focus on the subset where SGD is more likely to converge to.

3.1 Invariances in neural network function class

We say that a network is *invariant* with respect to a transformation if and only if the network resulting from the transformation represents the same function as the original network. There are two well-known invariances: one is the unit-rescaling due to positive homogeneity of ReLU activations (Neyshabur et al., 2015) and the other is permutation of hidden units. Unit-rescaling has been well-studied and empirical evidence suggests that implicit bias of SGD would make the solution converge to a stage where the weights are more



Figure 4: **Effect of architecture choice and task difficulty on barrier size.** Each row in Figure 4a and Figure 4b shows the effect of task difficulty while each column represents the effect of architecture on a specific dataset. Figure 4c notes that a pair of (architecture, task) has lower barrier if the test error is lower. Therefore, any changes in the architecture or the task that improves the test error, also improves the loss barrier. Effect of depth is stronger than (architecture, task) which leads to high barrier values for ResNets on MNIST, SVHN, CIF R10, CIF R100, and ImageNet.

balanced (Neyshabur et al., 2015; Wu et al., 2019). Since we are interested in the loss landscape through the lens of SGD and SGD is much more likely to converge to a particular rescaling, consideration of this type of invariance does not seem useful. However, in the case of permutations, all permutations are equally likely for SGD and therefore, it is important to understand their role in the geometric properties of the landscape and its basins of attraction.

Here we consider invariances that are in form of permutations of hidden units in each layer of the network, *i.e.*, each layer i with parameters W_i is replaced with $P_i W_i P_{i-1}$ where P_i is a permutation matrix and $P_i = P_0$ is the identity matrix. Note that our results only hold for permutation matrices since only permutation commutes with nonlinearity. We use $\mathcal{P}$ to refer to the set of valid permutations for a neural network and use π to refer to a valid permutation.

3.2 Our Conjecture

As mentioned above, SGD’s implicit regularization balances weight norms and, therefore, scale invariance does not seem to play an important role in understanding symmetries of solutions found by SGD. Consequently, here we focus on permutation invariance and conjecture that taking it into account allows us to have a much simpler view of SGD solutions. We first state our conjecture informally:

Most SGD solutions belong to a set $\mathcal{S}$ whose elements can be permuted in such a way that there is no barrier on the linear interpolation between any two permuted elements in $\mathcal{S}$.

The above conjecture suggests that most SGD solutions end up in the same basin in the loss landscape after proper permutation (see Figure 1 left panel). We acknowledge that the above conjecture is bold. Nonetheless, we argue that coming up with strong conjectures and attempting to disprove them is an effective method for scientific progress. Note, our conjecture also has great practical implications for model ensembling and parallelism since one can average models that are in the same basin in the loss landscape. The conjecture can be formalized as follows:

Conjecture 1. *Let $f(\theta)$ be the function representing a feedforward network with parameters $\theta \in \mathbb{R}^k$, $\mathcal{P}$ be the set of all valid permutations for the network, $P : \mathbb{R}^k \times \mathcal{P} \rightarrow \mathbb{R}^k$ be the function that applies a given permutation to parameters and returns the permuted version, and $B(\cdot, \cdot)$ be the function that returns barrier value between two solutions as defined in Equation 1. Then, there exists a width $h > 0$ such that for any*

network $f(\theta)$ of width at least h the following holds: There exist a set of solutions $\mathcal{S} \subseteq \mathbb{R}^k$ and a function $Q : \mathcal{S} \rightarrow \mathcal{P}$ such for any $\theta_1, \theta_2 \in \mathcal{S}$, $B(P(\theta_1, Q(\theta_1)), \theta_2) \approx 0$ and with high probability over an SGD solution θ , we have $\theta \in \mathcal{S}$.

Next, we approach Conjecture 1 from both theoretical and empirical aspects and provide some evidence to support it.

3.3 theoretical result

In this section we provide elementary theoretical results in support of our conjecture. Although the theoretical result is provided for a very limited setting, we believe it helps us understand the mechanism that could give rise to our conjecture. Below, we theoretically show that Conjecture 1 holds for a fully-connected network with a single hidden layer at initialization. Proof is given in appendix D.

Theorem 3.1. *Let $f_{\mathbf{v}, \mathbf{v}'}(\mathbf{x}) = \mathbf{v}^\top \sigma(\mathbf{v}' \mathbf{x})$ be a fully-connected network with h hidden units where $\sigma(\cdot)$ is ReLU activation, $\mathbf{v} \in \mathbb{R}^h$ and $\mathbf{v}' \in \mathbb{R}^{h \times d}$ are the parameters and $\mathbf{x} \in \mathbb{R}^d$ is the input. If each element of $\mathbf{v}'$ is sampled uniformly from $[-1/\sqrt{d}, 1/\sqrt{d}]$ and each element of $\mathbf{v}$ and $\mathbf{v}'$ is sampled uniformly from $[-1/\sqrt{h}, 1/\sqrt{h}]$, then for any $\mathbf{x} \in \mathbb{R}^d$ such that $\|\mathbf{x}\|_2 = \sqrt{d}$, with probability $1 - \delta$ over $\mathbf{v}, \mathbf{v}'$, there exist a permutation such that*

$$|f_{\mathbf{v} + (1-\epsilon)\mathbf{v}'', \mathbf{v}'' + (1-\epsilon)\mathbf{v}'}(\mathbf{x}) - f_{\mathbf{v}, \mathbf{v}'}(\mathbf{x})| = \tilde{O}(h^{-\frac{1}{2d-4}})$$

where $\mathbf{v}''$ and $\mathbf{v}'$ are permuted versions of $\mathbf{v}'$ and $\mathbf{v}$.

Theorem 3.1 states that for wide enough fully-connected networks with a single hidden layer, one can find a permutation that leads to having no barrier at random initialization. Although, our prove only covers random initialization, we believe with a more involved proof, it might be possible to extend it to NTK regime (Jacot et al., 2018). We leave this for future work.

3.4 Direct empirical evaluation of Conjecture 1

Another possible approach is to use brute-force (BF) search mechanism and find the function Q for elements of $\mathcal{S}$. The factorial growth of the number of permutations with the size of hidden units in each layer hinders exhaustive search for a winning permutation π to linear mode connect $P(\theta_1, \pi)$ and θ_2 . Even for MLPs with just one hidden layer brute-force works in reasonable time up to 2^4 neurons only, forcing the search to examine $2^4! \approx 2 \cdot 10^{13}$ permuted networks. BF is not feasible even for modest size deep networks. For small networks, one can use BF to find permutations between different models (see E.3). However, small size networks are not the focus of this paper and Conjecture 1 specifically mentions that.

Given the size of search space, using a more advanced search algorithm can be useful. The issue with this approach is that since it relies on the strength of a search algorithm, if the search algorithm fails in finding the permutation, one cannot be sure about the source of failure being the search algorithm or nonexistence of a permutation that leads to no barrier.

3.5 Our model vs real world: an alternative approach

We propose the following approach to circumvent the above obstacles. We create a competing set $\mathcal{S}'$ (our model) as a proxy for set $\mathcal{S}$ (real world). Given an SGD solution $\theta_1 \in \mathcal{S}$, we define $\mathcal{S}' = \{P(\theta_1, \pi) | \forall \pi \in \mathcal{P}\}$. We know that set $\mathcal{S}'$ satisfies the conjecture. For set $\mathcal{S}'$, all points are known permutations of θ_1 . Therefore, one can permute all points to remove their barriers with θ_1 , i.e, for all $\theta_2 = P(\theta_1, \pi)$, one can use $Q(\theta_2) = \pi^{-1}$ to remove the barrier between θ_1 and θ_2 . Our goal is therefore to show that $\mathcal{S}$ is similar to $\mathcal{S}'$ in terms of barrier behavior.

Equivalence of $\mathcal{S}'$ and $\mathcal{S}$ in terms of barriers, means that if we choose an element θ in $\mathcal{S}$, one should be able to find permutations for each of other elements of the $\mathcal{S}$ so that the permuted elements have no

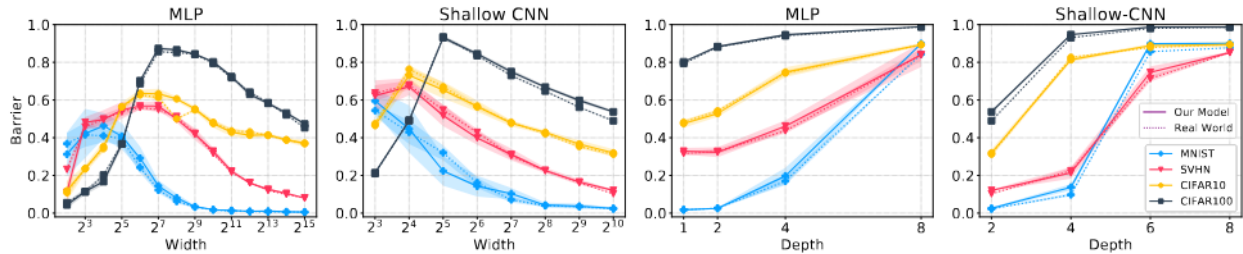


Figure 5: **Similar loss barrier between real world and our model *BEFORE* applying permutation.** From left to right: one-layer MLP, two-layer Shallow CNN, MLP and Shallow CNN with layer width of 2^1 . Increasing width first increases and then decreases the barrier, while adding more layers significantly increases the barrier size. We observe that $\mathcal{S}$ and $\mathcal{S}'$ behave similarly in terms of barrier as we change different architecture parameters such as width, depth across various datasets.

barrier with θ and hence are in the same basin as θ . The consequence of the equivalence of our model to real world is that Conjecture 1 holds. The conjecture effectively means that different basins exist because of the permutation invariance and if permutation invariance is taken into account (by permuting solutions to remove the barriers between them), there is only one basin, i.e., all solutions reside in the same basin in the loss landscape. We actually want to show $\mathcal{S}$ is similar to $\mathcal{S}'$ in terms of optimizing over all permutations but that is not possible so we show $\mathcal{S}$ is similar to $\mathcal{S}'$ in terms of barrier without search or when we search over a smaller set of permutations using a search algorithm. In the next section, we investigate our conjecture using this approach.

4 Empirical Investigation

In this section we show that $\mathcal{S}$ and $\mathcal{S}'$ have similar loss barrier along different factors such as width, depth, architecture, dataset and other model parameters (with and without searching for a permutation that reduces the barrier), hence supporting our conjecture. As discussed in Section 2, the barrier for both VGG and ResNet architectures is saturated at a high value hinting that the loss landscape might be more complex for these architecture families. In our experiments we observed that $\mathcal{S}$ and $\mathcal{S}'$ have similar high barriers for both of these architectures (see appendix E.2). Moreover, we observed that for both $\mathcal{S}$ and $\mathcal{S}'$ the employed algorithms (Section 4.2) were unable to find a permutation to reduce the barrier and hence our model shows a similar behavior to real world⁴. Given that the width and depth do not influence the barrier behavior in VGG and ResNet architectures, here we only focus on the effect of width and depth on barrier sizes for MLPs and Shallow CNNs.

4.1 Similarity of $\mathcal{S}$ and $\mathcal{S}'$

Figure 5 compares our model to the real world and shows that $\mathcal{S}'$ and $\mathcal{S}$ have strikingly similar barriers as we change different architecture parameters such as width and depth across various architecture families and datasets. This surprising level of similarity between our model and real world on variety of settings provide strong evidence for the conjecture. Even if the conjecture is not precisely correct as stated, the empirical results suggest that the structural similarities between our model and real world makes our model a useful simplification of the real world for studying the loss landscape. For example, the effect of width and depth on the barrier is almost identical in our model and the real world which suggests that permutations are perhaps playing the main role in such behaviors.

Algorithm 1 Simulated annealing (S_1) for Permutation Search

```
1: procedure  $S_1(\{\theta_i\}, i = 1..n, n \geq 2)$  ▷ Goal: minimize the barrier between  $n$  solutions
2:    $\pi_i = \pi_0, \forall i = 1..n$ 
3:   for  $k = 0; k < k_{\max}; k++$  do
4:      $T \leftarrow \text{temperature}(\frac{k+1}{k_{\max}})$ 
5:     Pick random candidate permutations  $\{\hat{\pi}_i\}, \forall i = 1..n$ 
6:     if  $\Psi(P(\theta_i, \hat{\pi}_i)) < \Psi(P(\theta_i, \pi_i))$  then ▷  $\Psi$ : barrier objective function
7:        $\pi_i \leftarrow \hat{\pi}_i$ 
   return  $\{\pi_i\}$ 
```

4.2 Search algorithms for finding a winning permutation

The problem of finding a winning permutation $\pi \in \mathcal{P}$ is a variant of the Travelling Salesman Problem where neurons are mapped to cities visited by a salesman. The problem belongs to the class of NP-hard optimization problems and *simulated annealing* (S_1) is often used to find a solution for such a combinatorial search problem. S_1 's performance however highly depends on the parameter choices, including the minimum and maximum temperatures, the cooling schedule and the number of optimization steps. The pseudocode of S_1 is shown in Algorithm 1. S_1 takes a set of solutions $\{\theta_i\}, i = 1..n, n \geq 2$ as input (we use $n = 5$) and searches for a set of permutations $\{\pi_i\}$ that reduce the barriers between all permuted $\binom{n}{2}$ solution pairs. To find the best $\{\pi_i\}$, in each step of S_1 the current candidate permutations $\{\hat{\pi}_i\}, i = 1..n$ are evaluated to minimize the objective function Ψ . We use two versions of simulated annealing that vary in their definition of Ψ to evaluate the conjecture.

Simulated annealing 1 (S_1). In the first version S_1 , Ψ is defined as the average pairwise barrier between candidate permutations $B(P(\theta_i, \pi_i), P(\theta_j, \pi_j)), i \neq j$.

Simulated annealing 2 (S_2). In the second version S_2 , we average the weights of permuted models $P(\theta_i, \pi_i)$ first and defined Ψ as the train error of the resulting average model. The simplest form of S_2 happens if $n = 2$ and is discussed in Section 3.

The rationale behind these two versions is that if the solutions reside in one basin, there is no barrier between them. Therefore averaging solutions in one basin yields another solution inside their convex hull.

Although each version of S has a different definition of the objective function Ψ to find the best permutation, for all S versions we report the average barrier between all pairs in the plots. Our empirical results suggest that S_1 and S_2 yield very similar performance. However, S_2 is significantly less computationally expensive, which makes it more suitable for exploring larger models. In the following sections we present the results obtained with S_2 only and refer to this version as S . For more details on S implementation see appendix 4.

The left two plots in Figure 6 show that S_2 is not able to find permutations that improve pair-wise barrier significantly. We know that S does not guarantee finding a solution and is known to lose its effectiveness on TSP benchmarks beyond 1'000 cities (Zhan et al., 2016). The effectiveness of S is also reduced here as we can only evaluate the cost of full route (divide and conquer is not possible). One way to increase this effectiveness is to reduce the search space which we will discuss next.

Search space reduction. In order to reduce the search space, here we only take two SGD solutions θ_1 and θ_2 , permute θ_1 and report the barrier between permuted θ_1 and θ_2 as found by S with $n = 2$. The right two plots in Figure 6 shows this intervention helps S to find better permutations. In particular, the barrier improves significantly for MNIST and SVHN datasets for both MLP and Shallow CNN across different width. However, similar to Section 2, we did not observe significant improvements when increasing depth (see Figure 12).

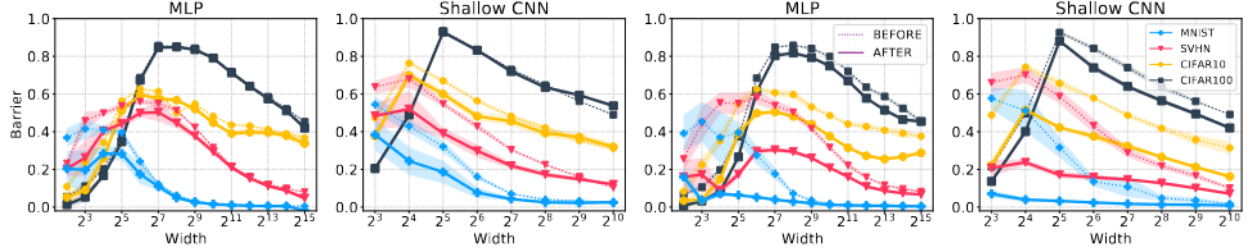


Figure 6: **Performance of Simulated annealing (S).** **Two Left:** S where we average the weights of permuted models first and ψ is defined as the train error of the resulting average model. **Two Right:** Search space is reduced *i.e.*, we take two SGD solutions θ_1 and θ_2 , permute θ_1 and report the barrier between permuted θ_1 and θ_2 as found by S with $n = 2$. When search space is reduced, S is able to find better permutations.

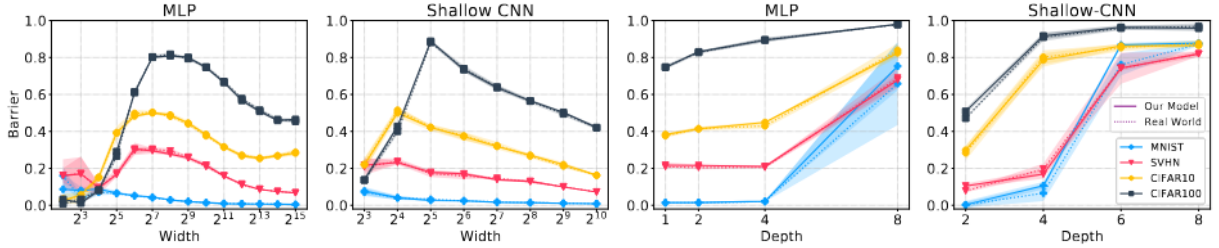


Figure 7: **Similar loss barrier between real world and our model F_{TER} applying permutation, when search space is reduced.** We observe that reducing the search space makes S more successful in finding the permutation to remove the barriers. Specifically, S could indeed find permutations that when applied to θ_1 result in zero barrier *e.g.*, MLP for MNIST where depth is 1 (across all width), 2 and 4 (where width is 2^1).

4.3 Similarity of S and S' after search

Figure 7 shows the surprising similarity of the barrier between S and S' even after applying a permutation found by a search algorithm (when search space is reduced). We also observe that reducing the search space makes S more successful in finding the permutation $\{\pi\}$ to remove the barriers. Specifically, in some cases S could indeed find permutations that when applied to θ_1 result in zero barrier. However, the fact that S' 's success shows a similar pattern for S, S' provides another evidence in support of the conjecture. For example, S successfully reduces the barrier for both S, S' on MNIST and SVHN datasets.

Figure 1 (right) summarizes our extensive empirical evidence (more than 3000 trained networks) in one density plot, supporting similarity of barriers in real world and our model across different choices of architecture family, dataset, width, depth, and random seed. Putting together, all our empirical results support our main conjecture.

5 Discussions and Conclusion

We investigated the loss landscape of ReLU networks and proposed and probed the conjecture that the barriers in the loss landscape between different solutions of a neural network optimization problem are an artifact of ignoring the permutation invariance of the function class. In a nutshell, this conjecture suggests that if one considers permutation invariance, there is essentially no loss barrier between different solutions and they all exist in the same basin in the loss landscape.

Our analysis has direct implication on initialization schemes for neural networks. Essentially it postulates that randomness in terms of permutation does not impact the quality of the final result. It is interesting to

⁴ If the experiments are included in the aggregated results reported in Figure 1.

explore whether it is possible to come up with an initialization that does not have permutation invariance and only acts like a perturbation to the same permutation. If all basins in the loss landscape are basically the same function, there will be no need to search all of them. One can explore the same basin while looking for diverse solutions and this makes search much easier and would lead to substantially more efficient search algorithms.

another area where our analysis is of importance is for ensembles and distributed training. If we can track the optimal permutation (that brings all solutions to one basin), it is possible to use it to do weight averaging and build ensembles more efficiently. Moreover, we are interested in answering the question whether there is a one-to-one mapping between lottery tickets and permutations. We believe our analysis laid the ground for investigating these important questions and testing the usefulness of our conjecture in ensemble methods and pruning, which is the subject of future studies.

The biggest limiting factor of our study is the size of the search space and hence we need a strong search algorithm, specially for deep models where the size and complexity of the search space was prohibitive in terms of computation for the existing search methods. We hope improvements of search algorithms can help us to extend these results. Lastly, our analysis focuses on image recognition task and extending the results to natural language tasks is of interest for future work.

acknowledgement

We would like to thank Guy Gur-ri and Ethan Dyer for their valuable discussions. This research was supported in part through computational resources provided by Google Cloud.

References

- Baldassi, C., Pittorino, F., and Zecchina, R. (2020). Shaping the learning landscape in neural networks around wide flat minima. *Proceedings of the National Academy of Sciences*, 117(1):161–170.
- Belkin, M., Hsu, D., Ma, S., and Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854.
- Brea, J., Simsek, B., Illing, B., and Gerstner, W. (2019a). Weight-space symmetry in deep networks gives rise to permutation saddles, connected by equal-loss valleys across the loss landscape. *arXiv preprint arXiv:1907.02911*.
- Brea, J., Simsek, B., Illing, B., and Gerstner, W. (2019b). Weight-space symmetry in deep networks gives rise to permutation saddles, connected by equal-loss valleys across the loss landscape.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: a large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee.
- Draxler, F., Veschgini, K., Salmhofer, M., and Hamprecht, F. (2018). Essentially no barriers in neural network energy landscape. In *International conference on machine learning*, pages 1309–1318. PMLR.
- Fort, S., Hu, H., and Lakshminarayanan, B. (2019). Deep ensembles: a loss landscape perspective. *arXiv preprint arXiv:1912.02757*.
- Fort, S. and Jastrzebski, S. (2019). Large scale structure of neural network loss landscapes.
- Frankle, J. and Carbin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks.
- Frankle, J., Dziugaite, G. K., Roy, D., and Carbin, M. (2020). Linear mode connectivity and the lottery ticket hypothesis. In *ICML*, volume 119, pages 3259–3269. PMLR.